



**UNIFAN – CENTRO UNIVERSITÁRIO NOBRE**  
**CURSO DE ENGENHARIA DA COMPUTAÇÃO**

**Especificação Técnica Da Linguagem: RoboticaLang**

DAVI LEÃO; ÍTALO ANDRADE SOUSA; JOÃO VICTOR PAIM SANTOS ALMEIDA;  
LUCAS NASCIMENTO DE ALMEIDA.

Feira de Santana, Bahia

Abril/2026



**UNIFAN – CENTRO UNIVERSITÁRIO NOBRE**  
**CURSO DE ENGENHARIA DA COMPUTAÇÃO**

**Especificação Técnica Da Linguagem: RoboticaLang**

DAVI LEÃO; ÍTALO ANDRADE SOUSA; JOÃO VICTOR PAIM SANTOS ALMEIDA;  
LUCAS NASCIMENTO DE ALMEIDA.

Documento técnico apresentado ao curso de Engenharia da Computação do Centro Universitário Nobre (UNIFAN), como requisito parcial para a avaliação da primeira etapa do TDE – Trabalho Discente Efetivo na disciplina de Compiladores.

**Docente:** Thiago Mariano.

Feira de Santana, Bahia

Abril/2026

## 1. Visão Geral

A **RoboticaLang** é uma linguagem de programação procedural, imperativa e fortemente tipada, projetada para a automação de processos lógicos. A linguagem utiliza uma sintaxe baseada na língua inglesa, terminadores de instrução explícitos (;) e definição de blocos estruturais por meio de indentação obrigatória.

## 2. Alfabeto e Regras Léxicas

A linguagem reconhece caracteres alfanuméricos (A-Z, a-z, 0-9) e símbolos especiais para operadores e delimitadores. A RoboticaLang é **case-sensitive**, distinguindo entre letras maiúsculas e minúsculas em identificadores e palavras-chave.

- **Comentários:** Suporte a comentários de linha única iniciados pelo caractere **#**. Todo conteúdo à direita deste símbolo é ignorado pelo analisador léxico.

## 3. Tipos de Dados e Variáveis

A linguagem suporta três tipos primitivos de dados:

1. **int:** Números inteiros de 32 bits.
2. **float:** Números reais com ponto flutuante.
3. **string:** Cadeias de caracteres delimitadas obrigatoriamente por aspas duplas (" ").

**Regras de Declaração e Atribuição:** As variáveis devem ser declaradas informando o tipo seguido pelo identificador. A atribuição é realizada pelo operador **=** e todas as instruções de atribuição devem ser finalizadas com ponto e vírgula (;).

## 4. Operadores e Expressões

A RoboticaLang permite a construção de expressões complexas utilizando parênteses para controle de precedência.

### 4.1. Operadores Aritméticos

- **+** (Soma)
- **-** (Subtração)
- **\*** (Multiplicação)
- **/** (Divisão)
- **%** (Resto da divisão)

### 4.2. Operadores de Comparação

- **==** (Igual a)

- **!=** (Diferente de)
- **>** (Maior que)
- **<** (Menor que)
- **>=** (Maior ou igual a)
- **<=** (Menor ou igual a)

### 4.3. Operadores Lógicos

- **and** (Conjunção lógica)
- **or** (Disjunção lógica)
- **not** (Negação lógica)

## 5. Estruturas de Controle de Fluxo

As condições dentro das estruturas de fluxo permitem a combinação de múltiplos operadores de comparação e múltiplos operadores lógicos em uma única sentença.

### 5.1. Condicionais

- **if/else:** Executa blocos de comandos baseados em uma condição booleana. O bloco "caso contrário" (**else**) é opcional.
- **switch/case:** Permite a múltipla escolha de caminhos de execução com base no valor de uma variável, incluindo uma cláusula **default** para situações não previstas nos casos anteriores.

### 5.2. Estruturas de Repetição (Loops)

- **Loop com contador (for):** Utilizado para iterações com número definido de execuções.
- **Loop sem contador (while):** Executa o bloco enquanto a condição lógica permanecer verdadeira.
- **Controle de Fluxo Interno:** Comandos **break** (interrupção imediata da repetição) e **continue** (salto para a próxima iteração do ciclo).

## 6. Sub-rotinas (Procedures e Functions)

A RoboticaLang diferencia sub-rotinas pela presença ou ausência de retorno de dados.

1. **Procedures:** Definidas para realizar um conjunto de instruções sem devolver um valor ao ponto de chamada. Podem ou não receber parâmetros.
2. **Functions:** Definidas para realizar cálculos ou lógicas que resultam em um valor. Devem obrigatoriamente retornar um dado e podem ou não receber parâmetros.